Kernell Abstraction

Description

Kernel

A central role of operating systems is protection, which involves isolating potentially untrustworthy applications and users to prevent them from damaging other applications or the operating system itself.

Protection Implementation

The implementation of protection is a task for the kernel of the operating system. The kernel, the lowest level of software running on the system, has full access to all the hardware of the machine.

The kernel must be trustworthy to perform any operation on the hardware. Everything else, meaning the untrusted software running on the system, runs in a restricted environment with limited access to the full power of the hardware.

Running Untrusted Code

In turn, the applications themselves often need to securely run untrusted third-party code.

An example is a web browser running embedded JavaScript to render a web page.

Without protection, a script with an embedded virus could take control of the browser, making users believe they are interacting directly with the web, while in reality, their web passwords are being forwarded to an attacker.

This design model — extensible applications that run third-party scripts — occurs in many different domains.

Applications become more powerful and widespread if third-party developers and users can customize them, but this raises the issue of how to protect the application itself from unauthorized extensions.

Performance and Protection

A key consideration is the need to provide protection while running the application code at high speed. The operating system kernel runs directly on the processor with unlimited rights. The kernel can perform any operation available on the hardware.

Application Execution

What about the applications?

They must run on the processor with all potentially dangerous operations disabled.

For this to work, the hardware must provide some assistance.

Shared Hardware Resources

Of course, both the operating system kernel and the application processes running with limited rights share the same machine: the same processor, the same memory, and the same disk. We keep in mind these two perspectives: when the operating system kernel is running, it can do anything; when running an application process on behalf of a user, the process's behavior is limited.

The Process Concept

- **Program vs. Process:**
  - *Program:* A static sequence of instructions.
  - *Process:* A dynamic entity that represents the execution of a program, running on the CPU with a limited set of privileges.
- **Process Control Block (PCB):**

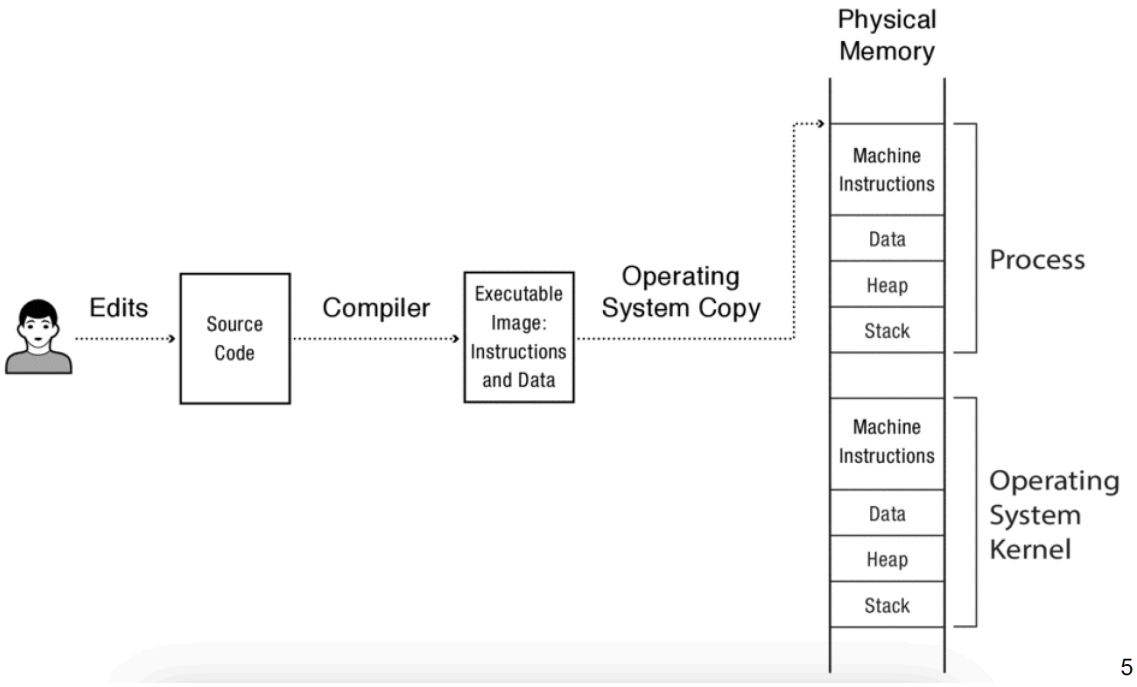
  The OS uses the PCB to track processes. It contains:
  - A unique process identifier (PID)
  - Process state (new, ready, running, waiting, terminated)
  - CPU register values, program counter (PC), and scheduling information (e.g., priority)

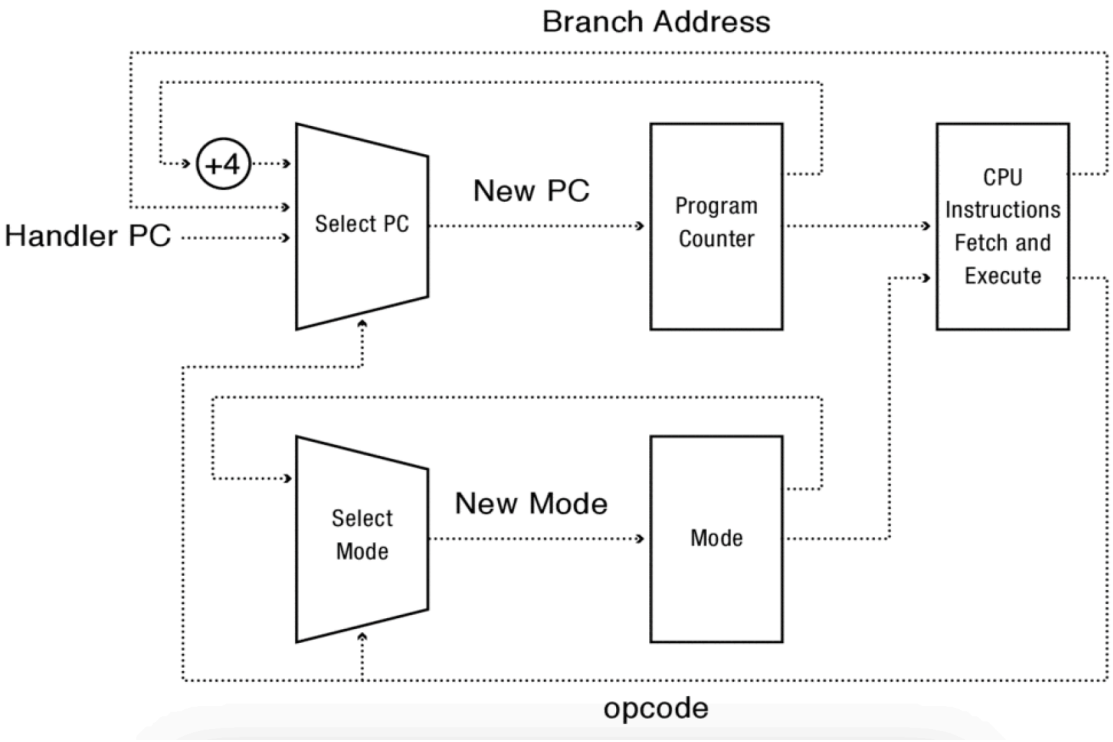
- Pointers to threads
- The process’s address space (memory limits and permissions)
- Other allocated resources such as open files and I/O status .

Dual-Mode Operation

- **Kernel Mode vs. User Mode:**
  - *Kernel Mode:* Runs with full privileges (access to any memory, I/O, disk sectors, etc.).
  - *User Mode:* Runs with restricted privileges defined by the OS.
- **Hardware Enforcement:**

  Mode bits stored in registers (e.g., EFLAGS on x86 or CPSR on ARM) control which instructions can be executed in user mode, ensuring that only kernel code can perform sensitive operations.

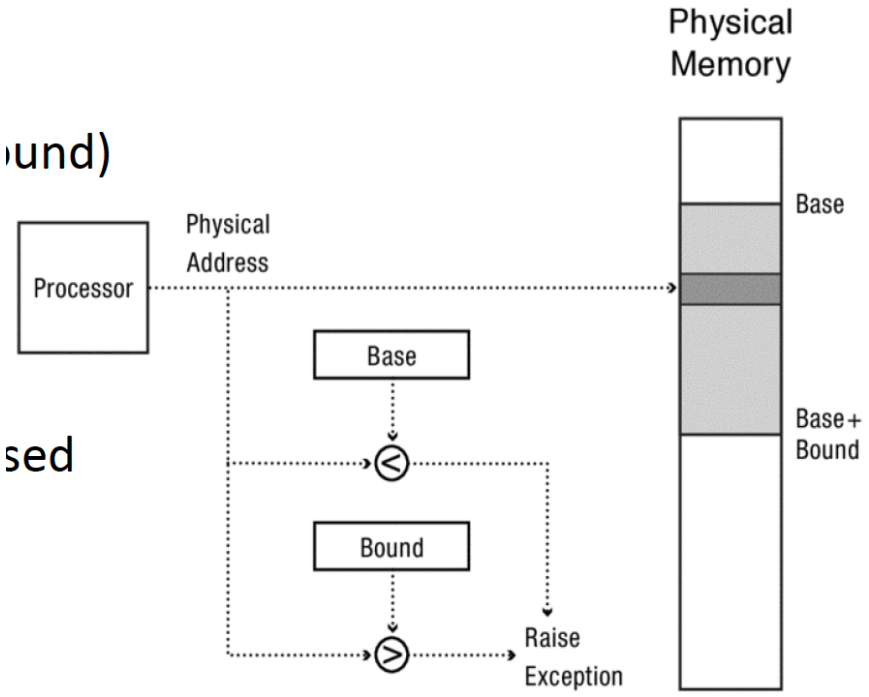




Memory Protection Mechanisms

- **Base-Bound Protection**:

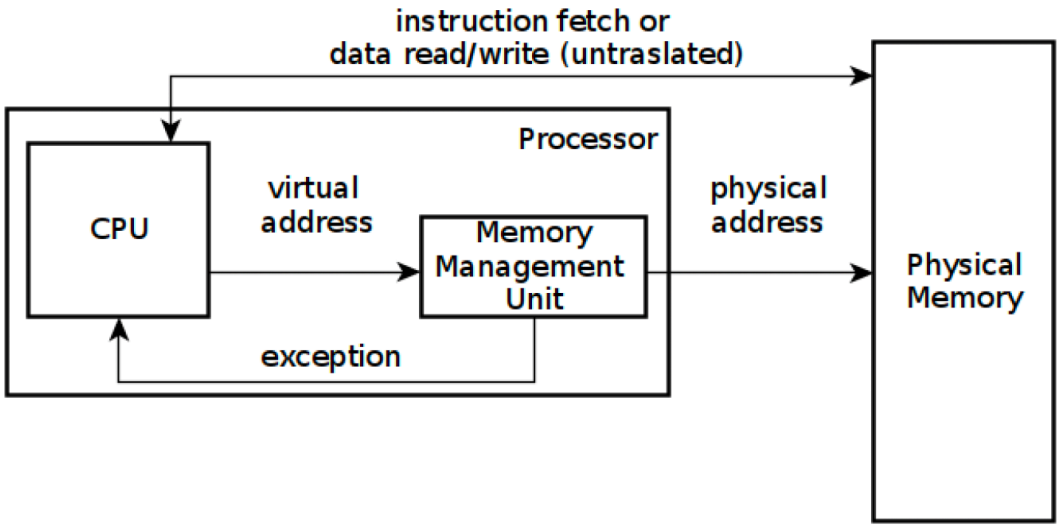
  Each process has a base and a bound register defining the valid range of physical memory addresses. Any memory access outside this range triggers an exception.



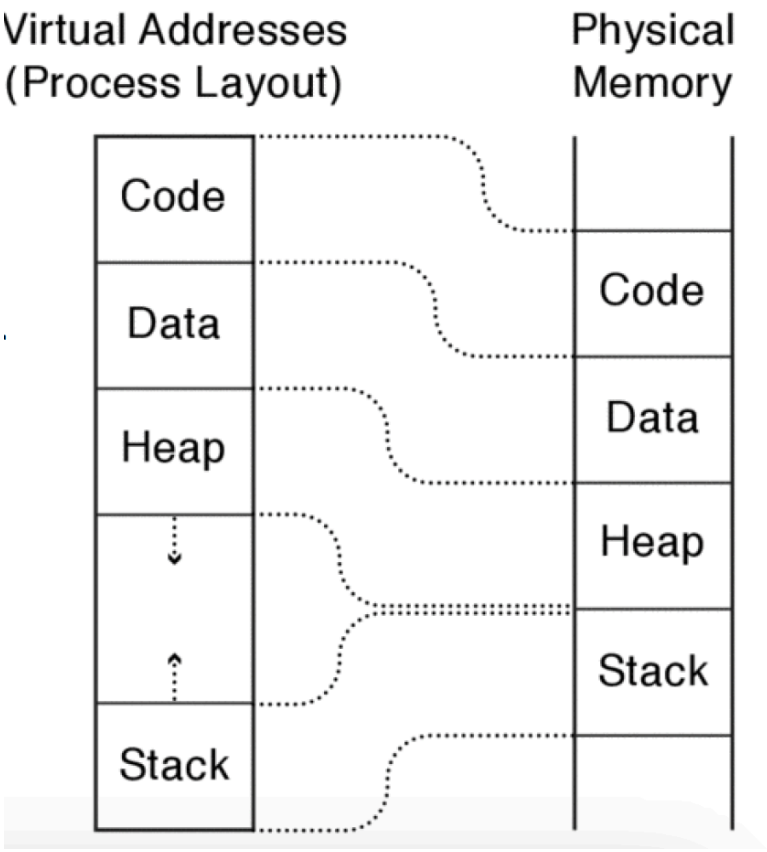
- **Virtual Memory and the MMU**:

  To overcome the limitations of base-bound protection (e.g., non-expandable heap/stack, difficulty in sharing memory), modern systems use:

  - **Virtual Addresses**: The hardware (via the Memory Management Unit) translates virtual addresses to physical ones.



- **Additional Benefits:** Support for shared code segments, dynamic libraries, memory-mapped files, and techniques like Address Space Layout Randomization (ASLR) .




```c
int staticVar= 1; // a static variable

main() {
    int localVar = 1; // a procedure local variable
    staticVar += 1;
    localVar += 1;
    sleep(10); // sleep causes the program to wait for 10
seconds
    printf("static address: %p, value: %d\n", &staticVar,
staticVar);
    printf("local address: %p, value: %d\n", &localVar, lo
```

```
calVar);
}
```

```
Produces (*):static address: 0x100407000, value: 2
local address: 0xffffcc3c, value: 2
```

Mode Switch and Safe Control Transfer

- **Switching from User to Kernel Mode:**

  Transitions occur via:

  - **Hardware Interrupts:** Triggered by devices or timers.
  - **Exceptions (Traps):** Resulting from errors (e.g., division by zero) or attempts to execute privileged instructions.
  - **System Calls:** When a user process requests a service from the OS via a controlled entry point.

- **Returning to User Mode:**

  After handling an interrupt, exception, or system call, the kernel restores the user process's state (PC, stack pointer, processor status word) and resumes execution in user mode.

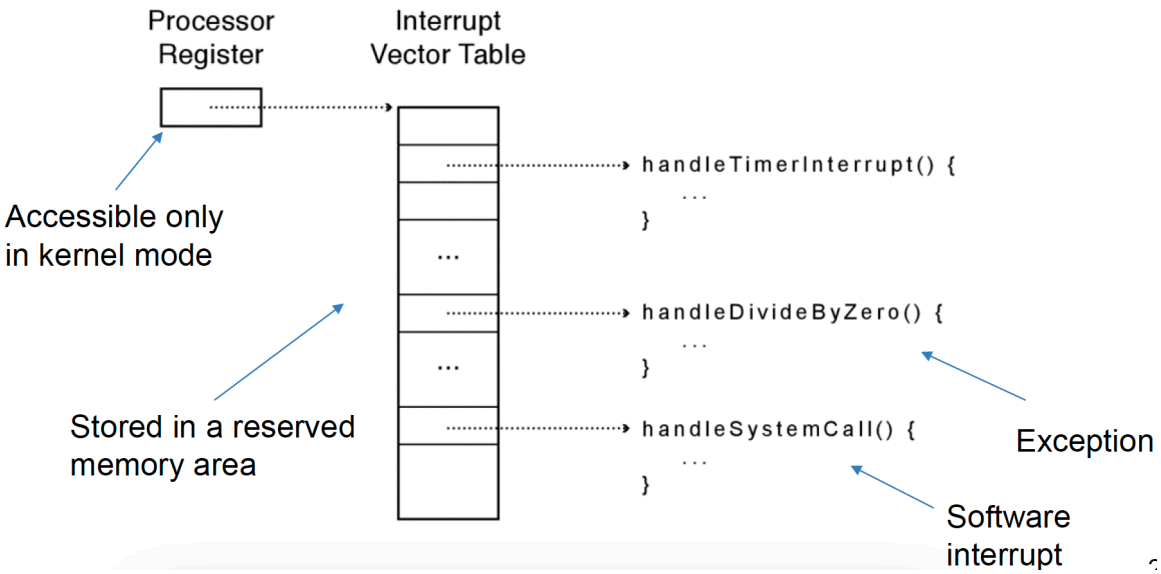
- **Atomic Transitions:**

  The change between modes must be atomic—saving and restoring critical registers (PC, SP, PSW) simultaneously—to avoid race conditions and ensure system integrity .

Interrupt Handling

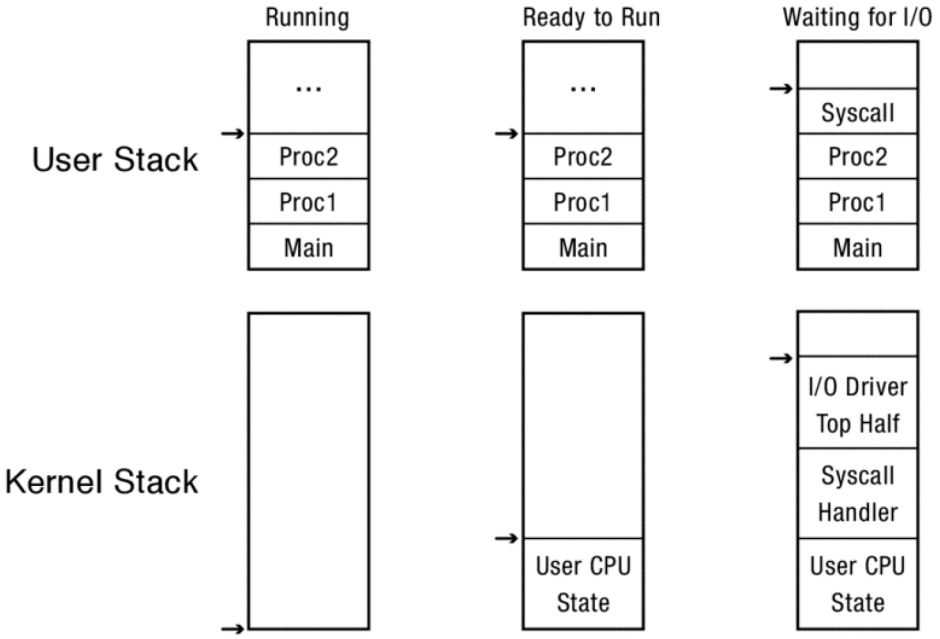
- **Interrupt Vector:**

  A table maintained by the OS that contains pointers to handlers for different types of interrupts (hardware interrupts, exceptions, software interrupts).



- **Dedicated Kernel Stack for Interrupts**:

  To protect the integrity of the user process's stack, interrupts are handled on a separate kernel stack.



- **Interrupt Masking**:

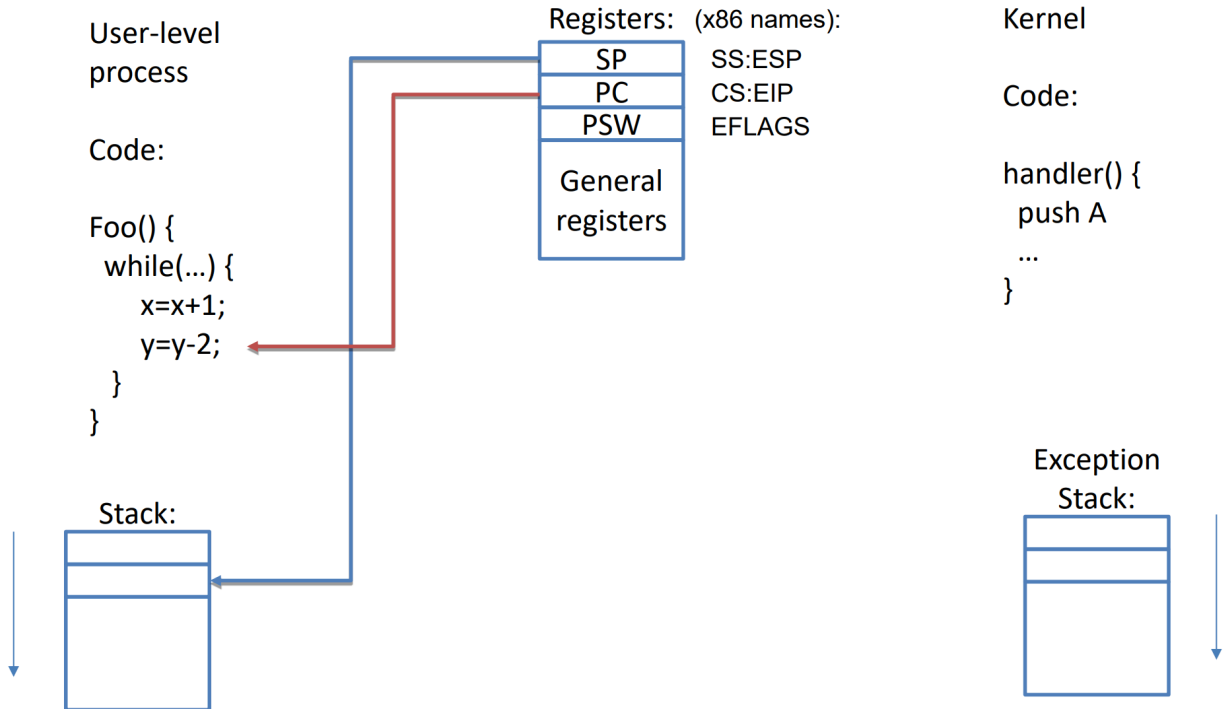
When an interrupt handler is invoked, interrupts are typically disabled (masked) to prevent nested interrupts until the handler completes.

- **Top-Half and Bottom-Half Handlers:**
  - *Top Half:* Quickly handles the immediate tasks of the interrupt, such as saving the state and acknowledging the device.
  - *Bottom Half:* Handles more extensive processing in a deferred manner, allowing the top half to finish quickly and re-enable interrupts .

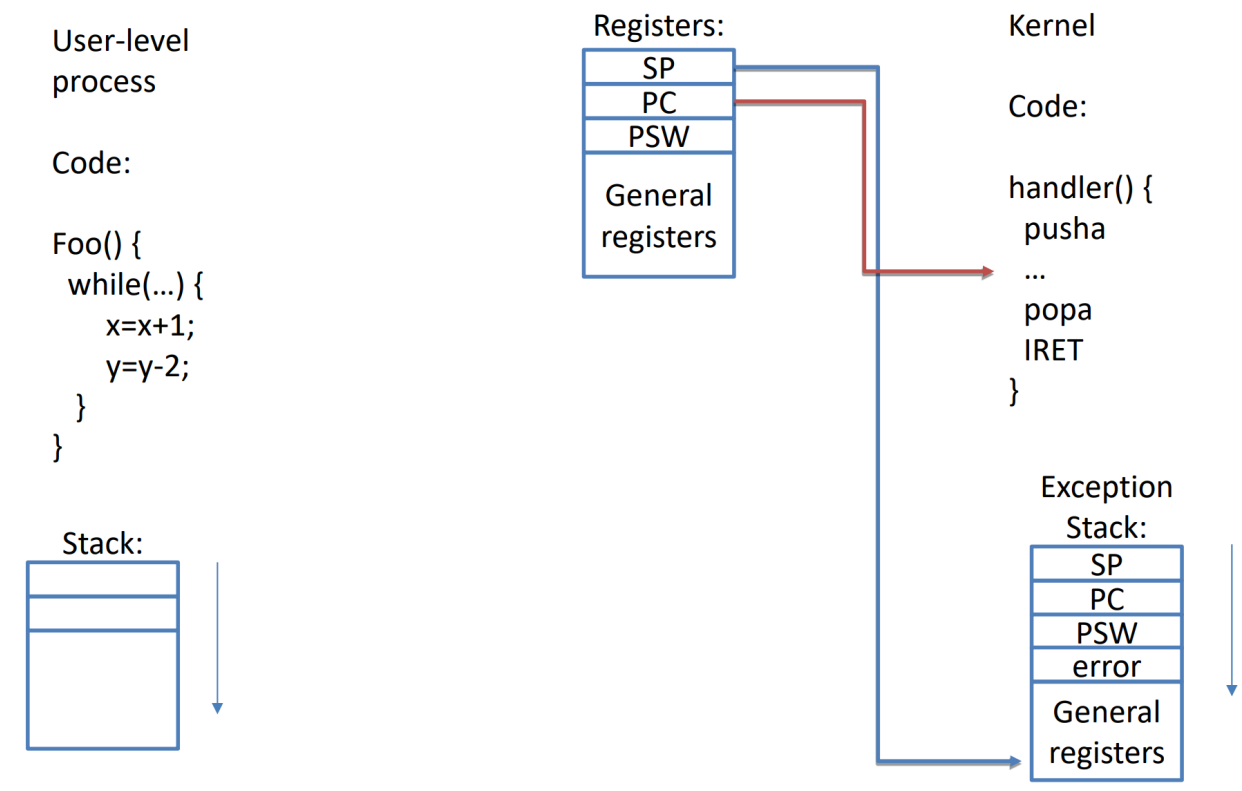
Atomic Mode Transfer

- **During an Interrupt (on *x86*):**
  - *The processor atomically saves the current stack pointer, program counter, and processor status word.*
  - *It switches from the user stack to the kernel stack.*
  - *The interrupt vector is used to jump to the appropriate handler.*
  - *The handler saves any additional registers it might modify.*
- **IRET Instruction:**The IRET (or equivalent) instruction restores the saved context atomically and returns control to the user process, ensuring that mode, PC, and SP are all correctly restored.

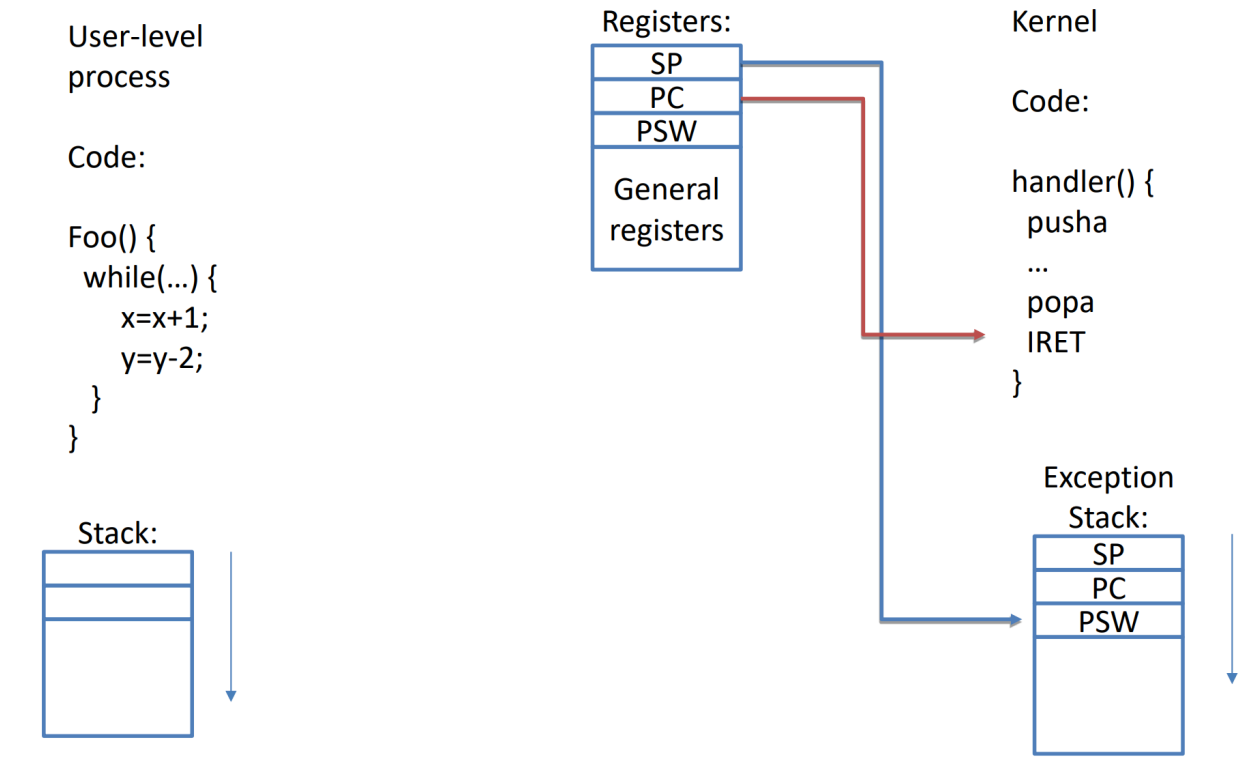
BEFORE

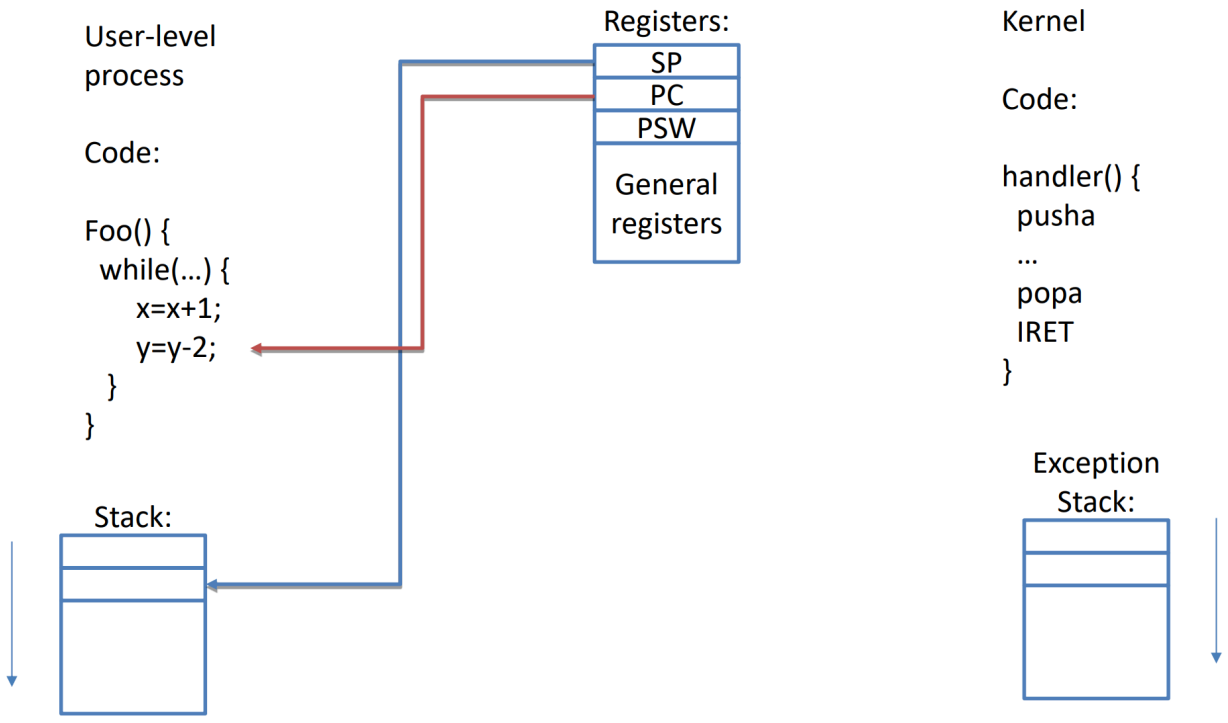

User-level
process
Code:
Foo() {
while(…) {
x=x+1;
y=y-2;
}
}
Stack:
Registers:
(x86 names):
SP
PC
PSW
General
registers
SS:ESP
CS:EIP
EFLAGS
Kernel
Code:
handler() {
push A
…
}
Exception
Stack:

DURING


User-level process
Code:
Foo() {
while(…) {
x=x+1;
y=y-2;
}
}
Stack:
Registers:
SP
PC
PSW
General registers
Kernel
Code:
handler() {
pusha
…
popa
IRET
}
Exception Stack:
SP
PC
PSW
error
General registers

AFTER





At end of handler:

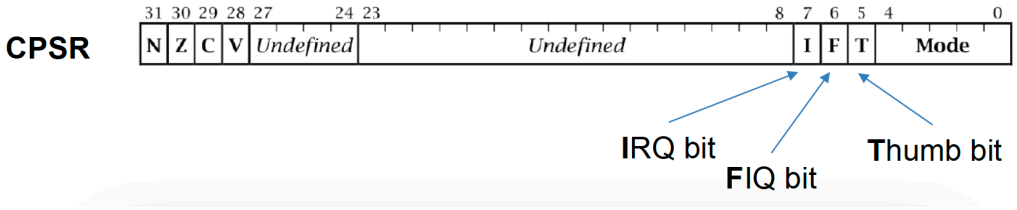
- Handler restores saved registers
- Atomically return to interrupted process/thread (IRET instruction in the previous example):
  - Restore program counter
  - Restore program stack
  - Restore processor status word/condition codes
  - Switch to user mode
  - Enable interrupts

Exception Handling in ARM

- **Multiple Modes and Banked Registers**:

  ARM processors support several modes (User, IRQ, FIQ, SVC, Undefined, Data Abort) with banked registers (e.g., separate SP, LR, CPSR for certain modes) to save context during exceptions.

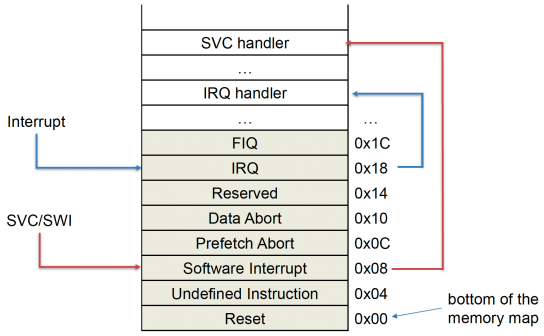
Mode	$CPSR_{4:0}$
User	10000
Supervisor	10011
Abort	10111
Undefined	11011
Interrupt (IRQ)	10010
Fast Interrupt (FIQ)	10001



- **Exception Vector Table**:

  Each exception type has a fixed address (e.g., Reset at 0x00, SVC at 0x08, IRQ at 0x18, FIQ at 0x1C) that contains a branch instruction to the actual handler.

Exception	Address	Mode
Reset	0x00	Supervisor
Undefined Instruction	0x04	Undefined
Supervisor Call	0x08	Supervisor
Prefetch Abort (instruction fetch error)	0x0C	Abort
Data Abort (data load or store error)	0x10	Abort
Reserved	0x14	N/A
Interrupt	0x18	IRQ
Fast Interrupt	0x1C	FIQ



- **Exception Handling Procedure**:
  - The banked link register (LR) is set to the return address (usually PC + 4).
  - The current CPSR is saved into the SPSR to preserve the pre-exception state.
  - The processor switches to the appropriate mode, maps in the banked registers, and disables further interrupts.
- **Returning from Exceptions**:

  Special instructions (e.g., MOVS, SUBS with the ‘S’ flag) restore the CPSR and the PC in an atomic operation to resume execution correctly

Banked registers:

User	System	Fast Interrupt	Interrupt	Supervisor	Abort	Undefined
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	**R8_fiq**	R8	R8	R8	R8
R9	R9	**R9_fiq**	R9	R9	R9	R9
R10	R10	**R10_fiq**	R10	R10	R10	R10
R11	R11	**R11_fiq**	R11	R11	R11	R11
R12	R12	**R12_fiq**	R12	R12	R12	R12
R13 (SP)	R13 (SP)	**R13_fiq**	**R13_irq**	**R13_svc**	**R13_abt**	**R13_und**
R14 (LR)	R14 (LR)	**R14_fiq**	**R14_irq**	**R14_svc**	**R14_abt**	**R14_und**
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)
Program Status Registers						
CPSR	CPSR	CPSR **SPSR_fiq**	CPSR **SPSR_irq**	CPSR **SPSR_svc**	CPSR **SPSR_abt**	CPSR **SPSR_und**

Fiq mode

FIQ Mode (Fast Interrupt Request Mode) is a special CPU mode in **ARM processors** designed to handle high-priority interrupts more efficiently than standard interrupts (IRQ - Interrupt Request).

Key Features of FIQ Mode

1. **Low Latency**: FIQ has a higher priority than IRQ and can interrupt an IRQ in progress.
2. **Dedicated Registers**: ARM provides separate general-purpose registers for FIQ, reducing the need to save and restore context, which speeds up execution.
3. **Faster Execution**: Since FIQ handlers use dedicated registers, they avoid the overhead of saving the global register context.
4. **Used for Critical Scenarios**: FIQ is commonly used in **real-time applications**, such as high-speed peripheral management (e.g., audio, video, networking).

FIQ Mode and Register Management in ARM Processors

When a **FIQ (Fast Interrupt Request)** is triggered, the ARM CPU enters **FIQ Mode**, a privileged execution mode with access to a set of **dedicated registers**. This allows the interrupt to be handled **faster** than IRQs by reducing the need for saving and restoring the context.

Registers in ARM Processors and FIQ Mode

The ARM architecture uses **banked registers**, meaning that different CPU modes can have their own versions of certain registers. In **FIQ Mode**, some registers are **banked**, so they are replaced with dedicated versions, preventing conflicts with code running in other modes.

Register	Normal Mode	FIQ Mode
R0 - R7	Shared	Shared
R8 - R12	Shared	**Banked (FIQ has its own registers)**
R13 (SP)	Stack Pointer	**Separate Stack Pointer for FIQ**
R14 (LR)	Link Register	**Separate Link Register for FIQ**
R15 (PC)	Program Counter	Shared
SPSR	Not available	**FIQ has its own SPSR**

How Does the Register Mechanism Work in FIQ Mode?

1. **FIQ Triggered**
   - When a FIQ is received, the CPU automatically switches to **FIQ Mode**.
2. **Use of Banked Registers**
   - Instead of saving and restoring general-purpose registers, the interrupt handler can directly use **R8-R14**, which are exclusive to FIQ.
   - This eliminates the need for push/pop operations, significantly improving speed.
3. **Separate Stack Pointer (`R13_fiq`)**

- FIQ Mode has its own **Stack Pointer (`R13_fiq`)**, ensuring that the user mode stack remains untouched.

4. **Separate Link Register (`R14_fiq`)**
   - When a FIQ is triggered, the **Program Counter (`PC`)** value is saved in `R14_fiq`, avoiding the risk of overwriting the normal `LR`.
5. **Returning from FIQ**
   - After handling the interrupt, the code executes `SUBS PC, R14_fiq, #4` to return exactly to the interrupted instruction.

IRQ

IRQ (*Interrupt Request*) is a mechanism used in **ARM processors** (and many other architectures) to handle external or internal events that require immediate attention from the CPU. IRQs allow the processor to pause its normal execution to respond to important events, such as keyboard input, data transfers, or timers.

How Does an IRQ Work?

1. **A device generates an interrupt request**
   - For example, a network card signals incoming data, or a timer notifies the end of a time interval.
2. **The CPU interrupts the current execution**
   - The CPU suspends the ongoing instruction and saves its current state (PC and CPSR) so it can resume later.
3. **Executes the IRQ handler**
   - The CPU runs a special routine called an **Interrupt Service Routine (ISR)** to process the event.
4. **Resumes normal execution**
   - After handling the interrupt, the CPU restores the saved state and continues executing the interrupted instruction.

Types of IRQs

Type of IRQ	Description
Hardware IRQ	Triggered by external peripherals (e.g., keyboard, mouse, network, disk)
Software IRQ (SWI)	Triggered by software to request operating system services
Timer IRQ	Triggered by an internal timer for periodic operations
External IRQ	Interrupt signals from external devices

IRQ vs. FIQ – What's the Difference?

Feature	IRQ (Interrupt Request)	FIQ (Fast Interrupt Request)
Priority	Lower	Higher
Banked Registers	No	Yes (R8-R14)
Response Time	Slower	Faster
Typical Usage	General-purpose interrupts	Critical events (e.g., audio/video)

Where is IRQ Used?

- **Operating systems (Linux, Windows, macOS)** to manage hardware events.
- **Embedded devices (Raspberry Pi, ARM microcontrollers)** for efficient hardware interaction.
- **Cybersecurity**: Analyzing interrupts can help detect rootkits or keyloggers.

Exception management

Reset:

```
R14_rst  = PC
SPSR_rst = CPSR
CPSR[4:0] = 0x10011 // supervisor mode
CPSR[5] = 0 // ARM state
CPSR[6] = 1 // Disable FIQ
CPSR[7] = 1 // Disable IRQ
PC = 0x00000000
```

SVC:

```
R14_svc  = PC+4  // next instruction
SPSR_svc = CPSR
CPSR[4:0] = 0x10011 // supervisor mode
CPSR[5] = 0 // ARM state
// CPSR[6] unchanged
CPSR[7] = 1 // Disable IRQ
PC = 0x00000008
```

IRQ:

```
R14_irq  = PC+4
SPSR_irq = CPSR
CPSR[4:0] = 0x10010 // IRQ mode
CPSR[5] = 0 // ARM state
// CPSR[6] unchanged
CPSR[7] = 1 // Disable IRQ
PC = 0x00000018
```

FIQ:

```
R14_fiq  = PC+4
SPSR_fiq = CPSR
CPSR[4:0] = 0x10001 // FIQ mode
CPSR[5] = 0 // ARM state
CPSR[6] = 1 // Disable FIQ
CPSR[7] = 1 // Disable IRQ
PC = 0x0000001C
```

Returning from an Exception

Interrupt received only after the execution of an instruction, thus PC already updated to point to the next instruction; LR banked contains PC+4, but PC instruction has not been executed. Therefore, LR must be decremented by 4

	Return instruction	ARM LR_x	THUMB LR_x
SWI	MOVS PC, LR_svc	PC+4	PC+2
UNDEF	MOVS PC, LR_und	PC+4	PC+2
FIQ	SUBS PC, LR_fig, #4	PC+4	PC+4
IRQ	SUBS PC, LR_irq, #4	PC+4	PC+4
PrefetchABT	SUBS PC, LR_abt, #4	PC+4	PC+4
DataABT	SUBS PC, LR_abt, #8	PC+8	PC+8
RESET	-	?	?

System Calls

- **Interface to Kernel Services:**

  System calls provide user processes with a controlled way to request privileged operations such as process management, file I/O, device control, and communication.

- **Mechanism:**
  - A system call is usually implemented via a software interrupt (SWI/SVC), which causes the CPU to switch into kernel mode.
  - The system call handler copies arguments from the user stack or registers, validates them, executes the requested service, and then copies results back to user space.
- **Example (ARM Linux):**

  The handler saves registers (using instructions like STMFD), looks up the system call in a table, performs necessary sanity checks, and then returns

using an instruction (MOVS PC, LR) that restores the CPSR.

Different types of SCs

- Process control (e.g., create/terminate processes)
- File management (e.g., open/close/read/write..)
- Device management (e.g., ioctl/read/write...)
- Information maintenance (e.g., getpid/gettimeofday/...)
- Communication (e.g., pipe/socket/shmget/...)
- Protection and security (e.g., chown/umask/setuid/...)

Upcalls and Signals

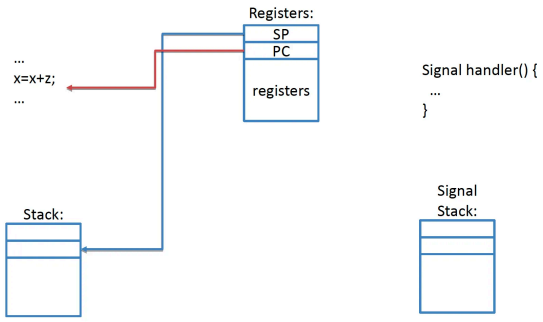
- **Upcalls (User-Level Interrupts):**

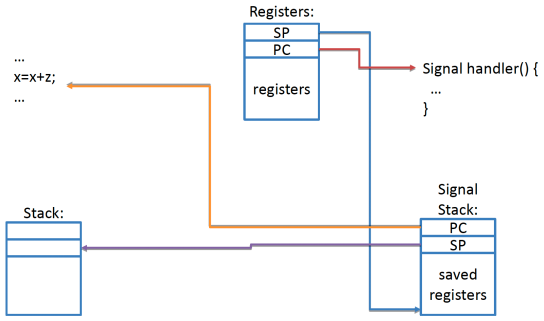
  Often implemented as Unix signals, upcalls notify a user process asynchronously about events that require immediate attention (e.g., a time-slice expiration or a specific event).

- **Handling Signals:**
  - A dedicated signal stack may be used in user space.
  - The system automatically saves the current process context (registers, PC, SP) and invokes a fixed entry point in the user process.
  - Signal masking is employed to prevent reentrant signal handling during the execution of a signal handler.

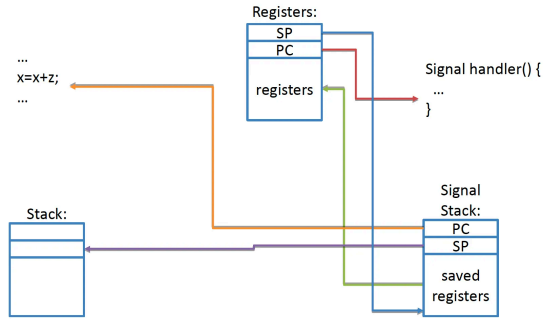
BEFORE:

DURING:

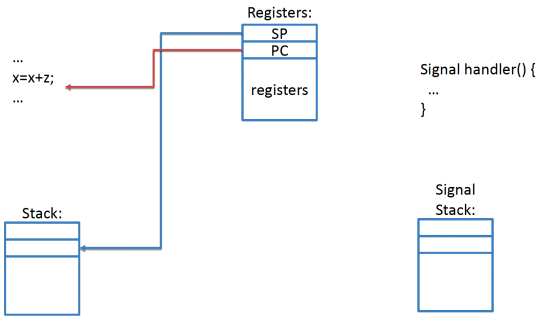




FINAL PHASE:

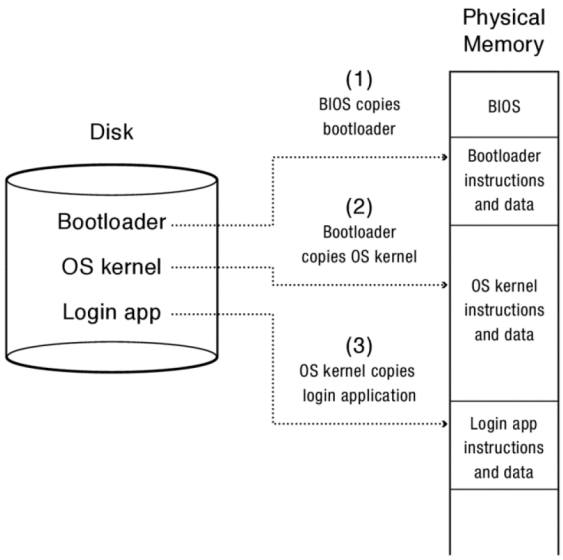


END OF HANDLER:



Kernel Booting

- **Boot Procedure:**
  - At power-on, the processor executes code in ROM (first-stage bootloader) that initializes basic hardware (e.g., memory controllers, I/O devices).
  - A second-stage bootloader is then loaded, which in turn loads the kernel and the root file system into memory.



- Control is transferred to the kernel, which then enables interrupts and begins normal operation .

Kernel System Call Handler:

Locate arguments:

- In registers or on user(!) stack
- Preliminary sanity checks

Copy arguments:

- From user memory into kernel memory
- Protect kernel from malicious code evading checks

Validate arguments:

- Protect kernel from errors in user code

Copy results back:

- into user memory

System Calls in ARM Linux:

The system call handler must know which kind of software interrupt is being called and where to find the arguments

Arch/ABI	Instruction	System call #	Ret val	Ret val2	Error
alpha	callsys	v0	v0	a4	a3
arc	trap0	r8	r0	-	-
arm/OABI	swi NR	-	a1	-	-
arm/EABI	swi 0x0	r7	r0	r1	-
arm64	svc #0	x8	x0	x1	-
blackfin	excpt 0x0	P0	R0	-	-
i386	int $0x80	eax	eax	edx	-

Arch/ABI	arg1	arg2	arg3	arg4	arg5	arg6	arg7
alpha	a0	a1	a2	a3	a4	a5	-
arc	r0	r1	r2	r3	r4	r5	-
arm/OABI	a1	a2	a3	a4	v1	v2	v3
arm/EABI	r0	r1	r2	r3	r4	r5	r6
arm64	x0	x1	x2	x3	x4	x5	-
blackfin	R0	R1	R2	R3	R4	R5	-
i386	ebx	ecx	edx	esi	edi	ebp	-

Summary: Hardware Support for Operating Systems

- **Privilege Levels**:

  The processor operates in at least two modes—user and kernel—to enforce a separation of privileges.

- **Privileged Instructions**:

Certain instructions are reserved for kernel mode. Attempts to execute them in user mode trigger exceptions.

- **Memory Protection**:

  Address translation via the MMU and base-bound methods prevents user processes from accessing or corrupting kernel data.

- **Interrupt and Exception Handling**:

  The processor uses timer interrupts, device interrupts, and exceptions to regain control from user processes when necessary.

- **Atomic Transitions**:

  Critical for safely switching between user and kernel modes, ensuring that the state (PC, SP, PSW) is restored without race conditions.

- **System Calls and Upcalls**:

  Provide interfaces for user processes to request kernel services and for asynchronous notifications (signals).

- **Boot Process and Virtualization Support**:

  The boot ROM loads startup routines into memory, and modern systems may also support virtualization through additional hardware privilege levels and atomic read-modify-write operations for synchronization.